Express

Rev 6.2 del 28/01/2026

Il framework **Express** 2
La gestione delle routes 3
Il parametro next ed il codice middleware 3
Regole di match delle routes 4
I response methods 5
Concetto di mount point 6
L'accesso alle risorse statiche del server 6
Accesso ai parametri GET 8
Accesso ai parametri POST 8
Passaggio dei parametri come risorsa 9
La gestione degli errori 10

I server **CRUD** 11
Accesso da smartphone e Hotspot mobile 12
CORS: Richieste Extra Domain 13
Creazione di un server **https** 18

Upload di un file binario 21
Richieste server to server 24
Invio di una **mail** 25

Gestione delle variabili **Session** 26
Approccio SSR ed utilizzo del template **EJS** 27

Express

E' un modulo aggiuntivo di NodeJs che permette di realizzare un server http.
Rispetto all'utilizzo diretto di Node presenta due vantaggi importanti :

- Presenza di un **dispatcher** integrato, che nella terminologia di Express si chiama **`router`** e che ha come compito quello di servire le richieste provenienti dal client.
- Disponibilità di molte librerie aggiuntive ad hoc per express (ad esempio cors, body-parser, etc)

Esiste anche una utility **`express – generator`** che consente di creare automaticamente un web site completo basato su **`express`** e su **`jade`** che è un template per scrivere rapidamente pagine html che saranno poi convertite in html vero e proprio da jade stesso.

Utilizzo di express

```
npm install express
npm install -D  @types/express   // per la definizione implicita dei tipi

let express = require('express');
let app = express();

const server = http.createServer(app);
server.listen(port, [ipAddress], callback())
```

L'oggetto **`app`** restituito dal costruttore express() è unico per l'intera applicazione ed è una **classe**, cioè sostanzialmente una funzione che però presenta anche Proprietà e Metodi.

Nel dettaglio app è una funzione definita nel seguente modo:

```
let app = function(req, res, next) {
       handle(req, res, next){   }
}
```

all'interno della quale vengono poi definiti diversi metodi, fra cui

```
  this.use = function(resource, callback){      }
  this.get = function(resource, callback){      }
  this.post = function(resource, callback){      }
```

i quali sono tutti metodi che consentono di aggiungere una route (identificata tramite resource e calback) all'interno di un apposito **vettore enumerativo** di routes gestito internamente da express.

app sostanzialmente è la funzione di callback attesa da **`http.createServer()`** che verrà eseguita in corrispondenza dell'arrivo di ogni richiesta (esattamente come in node base). A questa funzione vengono iniettati tre parametri relativi a richiesta e risposta, cioè **`req res next`**. Ogni volta che arriva una richiesta la funzione **`handle(req, res, next)`** esegue la prima route che fa match con la url della richiesta.

server.listen() avvia il server sulla porta indicata. La funzione di callback viene richiamata solo in corrispondenza dell'avvio e serve per eventuali inizializzazioni.
Si potrebbe anche usare direttamente ***`app.listen()`*** che però non consente ulteriori configurazioni.

Se due richieste arrivano in parallelo, la seconda viene accodata finchè non termina l'esecuzione della prima. Alcuni web server, come heroku e render, consentono di 'acquistare' più web worker paralleli che possono servire più richieste parallelamente. Ogni web worker avrà una propria copia indipendente dell'istanza app. Un apposito proxy (load balancer) distribuirà le richieste a turno (round-robin), sulla base del carico e tempo di rsposta stimato. E quindi avvierà il worker opportuno.
Notare che occorre evitare di usare variabili globali che NON sono condivise tra i vari worker.

La gestione delle routes

Per ogni **route** occorre definire :

- **resource** cioè la risorsa di ascolto. **Path** e **Resource** sono sinonimi
- **handler** (cioè la funzioni di callback da eseguire come risposta alla richiesta)

In corrispondenza dell'arrivo di una richiesta, express 'scorre' sequenzialmente l'elenco delle routes fermandosi automaticamente alla prima route che va a buon fine, cioè che corrisponde per il metodo (get, post) e per il nome della risorsa ("/api/elencoUser"). Quando trova la route che soddisfa questi criteri esegue la callback corrispondente.
Le routes vengono accedute in sequenza, per cui è fondamentale l'ordine con cui vengono scritte.

app.get(resourse, callback(req, res, next)
Crea un listener per la gestione della risorsa GET indicata.

app.post(resourse, callback(req, res, next)
Crea un listener per la gestione della risorsa POST indicata

app.patch(resourse, callback(req, res, next)
Crea un listener per la gestione della risorsa PATCH indicata

app.route('/risorsa1')
consente di raccogliere in un unico contenitore metodi di risposta diversi (get, post, put, delete, etc) relativamente ad una medesima risorsa:

```
.get ( function(req, res) {res.send('processing the get request ')} )
.post( function(req, res) {res.send('processing the post request')} );
```

app.use(resource, callback(req, res, next) Crea un listener per la gestione della risorsa indicata, listener che verrà eseguito **sempre**, indipendentemente dal ***metodo*** utilizzato per la richiesta.

In tutti i metodi il parametro **resource** può assumere il valore **"*"** che ha il significato di "qualsiasi risorsa". In pratica il listener viene eseguito in corrispondenza di qualsiasi richiesta.

Il parametro next ed il codice middleware

Nel momento in cui una **route** va a buon fine, express la esegue **ed interrompe la ricerca** a meno che, al termine della procedura, non venga richiamato il metodo **next(),** quello passato come terzo parametro che è un puntatore ad una funzione interna di express che fa ripartire la scansione dalla route successiva.

Le routes che utilizzano un next() finale, nel linguaggio di express sono indicate come **middleware (codice intermedio), e consentono di eseguire del codice intermedio comune a tutte le richieste,** come ad esempio stampare dei log o eseguire altre elaborazioni. Esempio:

```
app.use("*", function (req, res, next) {
   console.log(req.method + " : " + req.originalUrl);
   next();
});
```

Una route:

- **o esegue next() (nel qual caso si parla di 'middleware')**
- **oppure invia una risposta al client e termina la scansione (client route)**

next() richiamato **senza parametri** rimanda alla route successivva
next(err) rimanda direttamente alla route di errore,

NON è però possibile indicare esplicitamente la specifica route a cui passare.

Nel caso in cui nessuna **route** vada a buon fine si può aggiungere in coda una **default route** generica finale che provveda a restituire il tipico 404 'not found'

```
app.use("*", function (req, res, next) {
    res.status(404);
    res.send("Sorry cant find that! ");  });
```

In realtà se la scansione arriva al termine del file senza eseguire nessuna end()/send(), express invierebbe automaticamente un messaggio di errore del tipo : **Cannot GET/POST resource**

Regole di match delle routes: differenze tra app.use() e app.get()/app.post()

Mentre lo scopo di app.get(), app.post(), etc è quello ri **rispondere alle richieste del client**, lo scopo di app.use() è quello di eseguire codice **middleware**, che solitamente deve essere eseguito indipendentemente sia dal **metodo** di richiesta, sia dalla **risorsa** richiesta, per cui:

- **app.use()** risponde a su tutti i metodi di chiamata (get, post, put, delete, patch)
- **app.use()** a differenza di app.get() e app.post() che eseguono un match perfetto sulla risorsa richiesta, **app.use non esegue un match esatto sulla risorsa richiesta, ma va a buon fine anche nel caso in cui la richiesta riguardi una qualsiasi sottorisorsa**. Ad esempio :
  app.use('/apple', ..) will match '/apple', '/apple/img', '/apple/img/news'

  In quest'ottica la richiesta app.use('/') va sempre a buon fine e, nel caso di app.use(), "/" diventa quasi equivalente a "*".
  "/" si usa nel caso delle routes di tipo middleware che NON eseguono un match esatto
  "*" si usa nel caso delle routes di risposta (route esplicite: get, .post, patch, put, delete)
- **app.use()** **consente di omettere la risorsa**. L'omissione della risorsa equivale a **"/"** (tutte le risorse) In app.get() e app.post() la risorsa NON PUO' ESSERE OMESSA. Al limite si usa un "*****".

Nota: Per tutte le routes, come 1° param, invece di una singola risorsa, si può passare un vettore di risorse

Concetto di mount point

La risorsa per la quale è in ascolto un listener è detta mount point della route.

Nel caso di node, la proprietà req.url conteneva sempre il path completo richiesto dal browser.
express invece realizza una gestione più fine delle url. Supponimo che il browser richieda la seguente url:
'http://www.example.com**/admin/files/news**'

Nel momento in cui incontra una route, express confronta il **path** richiesto dal browser con il **path** di ascolto della route, e va a settare all'interno di **req** tre proprietà distinte:

req.originalUrl contenente la url completa richiesta dal client **/admin/files/news'**
req.baseUrl contenente il mount point di ascolto della route, cioè la risorsa su cui è in ascolto app.use()
req.url contenente la porzione di risorsa eccedente rispetto al mount point, cioè la differenza tra **originalUrl** (path completo) e **baseUrl** (mount point)

In pratica : **originalUrl = baseUrl + url**

Esempio:

```
app.use('/admin/files', function(req, res, next) {
    console.log(req.originalUrl);    /admin/files/news
    console.log(req.baseUrl);        /admin/files
    console.log(req.url);            /news  (equivalente a req.path)
    next();
});
```

Se si specifica come risorsa di ascolto `'/'`

`req.orignalUrl`	risorsa completa richiesta dal client
`req.baseUrl`	stringa vuota
`req.url`	risorsa completa

Se invece si specifica come risorsa di ascolto `'*'`

`req.orignalUrl`	risorsa completa richiesta dal client
`req.baseUrl`	risorsa completa richiesta dal client
`req.url`	stringa vuota

Poiché i middleware di express (fra cui ad es **express.static**) utilizzano di solito la proprietà **`req.url`**, per cui tra / e * **occorre utilizzare sempre /,** in modo che **`req.url`** restituisca la risorsa completa.

Accesso alle risorse statiche : express.static()

All'interno di express esiste un unico built-in middleware costituito dal metodo **`express.static()`** per la ricerca e restituzione delle risorse statiche. Nelle versioni successive è stato integrato anche il bodyParser per la lettura dei parametri Post.

I file statici (immagini, css, js, favicon.ico) devono essere inseriti all'interno di una cartella arbitraria (es **`public o static`**) indicata ad express come parametro del middleware **`express.static()`** che accetta indifferentemente sia path assoluti che relativi. Le seguenti righe sono del tutto equivalenti :

```
app.use('/', express.static('./static') );
app.use('/', express.static('static') );
app.use('/', express.static(__dirname + '/static') );
app.use('/', express.static(".")); // file client nella stessa cartella del server
```

Il metodo **express.static**()verifica se, all'interno delle cartella indicata, esiste la risorsa richiesta.

- **Se esiste**, la restituisce al client senza eseguire il metodo next.
- **Se invece non esiste** esegue il metodo **next()** e prosegue nella scansione delle routes dinamiche

Si possono specificare in sequenza più middleware in ascolto su sottocartelle differenti.
Il secondo path viene aggiunto al primo senza sovrascriverlo.

```
app.use(express.static('./cartella1'));
app.use(express.static('./cartella2'));
```

<u>Il primo parametro</u> (facoltativo) indica al solito la risorsa di ascolto.

/	la route risponderà in corrispondenza di qualsiasi richiesta
/static	la route risponderà **soltanto** per quelle risorse che cominciano con `/static` (prefisso che verrà però eliminato dal router come indicato nella seguente nota (4)

Struttura del middleware static

I middleware sono normalmente strutturati nel modo seguente (***<u>pseudocodice</u>*** di *`express.static`*)

```
express.static = function (root) {
  return function(req, res, next) {
    const filePath = join(root, req.path);
    if(filePath exists)
       read and return file
    else
       next()
  }
```

Note

1) Se il client richiede /, il listener statico cerca automaticamente il file **`index.html`**.
Se lo trova lo invia altrimenti richiama next().

2) Il metodo express.static() risponde soltanto in corrispondenza **delle richieste GET e HEAD**, che, secondo il protocollo http, sono le uniche abilitate a richiedere risorse statiche.

3) express.static potrebbe anche essere richiamata nel seguente modo:

```
app.use("/", (req, res, next) => {
  express.static("./static")(req, res, () => {
    if (!res.headersSent) // se non è stata spedita nessuna risposta
       next();
  });
});
```

cioè express.static(), al termine della gestione della richiesta, invoca la funzione che gli viene passata come terzo parametro (il "next"). In questo caso, invece di passargli il classico next del middleware principale, gli viene passata una funzione anonima personalizzata definita localmente.

A differenza della maggior parte dei middleware, express.static() **invoca sempre** questa funzione di completamento, **indipendentemente dall'esito della ricerca del file**, e lo fa **solo dopo aver concluso l'eventuale invio della risposta al client**.

Per questo motivo, all'interno di questa funzione di callback personalizzata è necessario verificare se la risposta è già stata inviata oppure no. Se non è stato spedito nulla, allora si deve chiamare next() per consentire al flusso middleware principale di proseguire.

4) Il primo parametro ha il significato di **`baseUrl`**, mentre la risorsa richiesta dall'utente è memorizzata in forma completa all'interno di **`originalUrl`**.
Il campo utilizzato da express.static() per effetuare la ricerca è il campo **url**, ottenuto come differenza tra originalUrl e baseUrl.

- Utilizzando **`baseUrl`** = "**`*`**" **url** sarà vuoto per cui il listener, in corrispondenza di ogni richiesta successiva proveniente dalla pagina.html, continuerà sempre a restitire index.html !!
- Utilizzando **`baseUrl`** = "**`/`**", la proprietà **url** coincide con la richiesta del client, per cui la risorsa viene individuata correttamente.

Sostanzialmente baseUrl rappresenta un punto di montaggio virtuale (**`mount_path`**) della cartella statica. Il client dovrà anteporre questo prefisso ad ogni richiesta; in corrispondenza di ogni richiesta il server provederà automaticamente a togliere tale prefisso

Si supponga di avere all'interno di static una sottocartella images. Per richiedere una immagine si potrà utilizzare una delle seguenti sintassi tutte equivalenti

1) `app.use('/', express.static('static'));`
`http://localhost:1337/images/desert.jpg`
Il file **`images/desert.jpg`** viene cercato all'interno della cartella **`./static`**

2) `app.use('/images', express.static('static'));`
`http://localhost:1337/images/images/desert.jpg`
il listener andrà a cercare `images/desert.jpg` all'interno della cartella **`./static`**.
Il primo `images` viene automaticamente eliminato dalla url di ricerca (originalUrl - baseUrl)

3) `app.use('/images', express.static('static/images'));`
`http://localhost:1337/images/desert.jpg`
Il primo `images` viene automaticamente eliminato.
Il listener andrà a cercare il file `desert.jpg` all'interno della cartella **`./static/images`**

4) Se index.html richiama una pagina2.html che si trova in una sottocartella **/pagine/pagina2.html**, quando poi pagina2.html a sua volta richiama le sue risorse es
`<link rel="stylesheet" href="**css/pagina2.css**" />)`
il browser **automaticamente** antepone alla richiesta **/pagine** che era stato utilizzato per richiedere pagina2.html per cui la richiesta inviata al server sarà **/pagine**/`/css/pagina2.css`

Lato server la ricerca dei file statici continua a funzionare senza dover aggiungere nessuna route statica

middleware come function

Un middleware può anche essere scritto come semplice function senza utilizzare **`app.use()`** per renderlo poi utilizzabile soltanto all'interno di una o più routes specifiche:

```
const authMiddleware = function (req, res, next) {
  if (req.headers.authorization)
    next();
  else
    res.status(401).send("Non autorizzato");  };
app.get('/api/utente', authMiddleware, (req, res) => {
  res.send("Dati utente");
});
```

Metodi per l'invio della risposta al client

Nel caso di express, l'oggetto **`respsonse`** **espone alcuni metodi aggiuntivi** rispetto ai metodi standard esposti dall'oggetto httpResponse di nodejs.

res.writeHead(200, header) res.end() // stessi di nodejs

res.**send**(str) Prima di inviare la risposta al client imposta automaticamente :
- **`response-code = 200`**
- **`content-type`** sulla base del tipo di oggetto passato a send (nel caso di una stringa sarà text/plain, nel caso di un json sarà application/json)
- Nel caso di un json, al contrario di **end()** provvede automaticamente a serializzare la risposta
- A differenza di **`end()`** trasmette automaticamente anche i cookies

res.**status**(404) Consente di impostare esplicitamente lo stato della risposta da inviare con .send()

res.**set('Content-Type', 'text/html'**) Consente di impostare esplicitamente una intestazione http.
Analogo al setHeader() di nodejs.

res.sendStatus(code) Opposto al precedente. Invia una risposta fissa contenente il codice indicato e la sua stringa rappresentativa come body ("OK", "NOT FOUND"; "SERVER ERROR")

res.json() Simile a send() ma:
- serializza SEMPRE. send(msg) non serializza le stringhe (cioè i semplici msg)
- può formattare il json (che invia) tramite apposita formattazione con app.set()
  `app.set('json spaces', 2); // number of indentation spaces`

res.sendFile(file) Legge e Invia il contenuto di un intero file (trasmesso in formato bytes stream).
Durante l'apertura del file legge automaticamente anche il tipo del file e lo assegna alla property **Content-Type**
Il **path** deve necessariamente essere un **percorso assoluto**.
Esisteva una vecchia versione **`res.sendfile()`** scritta in minuscolo che accettava come parametro un percorso relativo, però deprecata.

```
app.use(function (req, res, next) {
        res.status(404);
        res.sendFile(__dirname + '/static/error.html')  });
```

__dirname è il percorso assoluto della cartella corrente (dove si trova server.js)

In alternative è possibile utilizzare anche la seguente sintassi, che si aspetta come secondo parametro un percorso relative :

```
res.sendFile('error.html', {"root": './static'});
```

Altri Response Methods

res.redirect()	Redirect a request.
res.jsonp()	Send a JSON response with JSONP support.
res.download()	Prompt a file to be downloaded.
res.render()	Render a view template.

Accesso ai parametri GET

- **i parametri GET, qualunque libreria si usi, alla fine devono sempre essere accodati alla URL**
- i parametri **GET** di tipo **numerico** e **booleano** vengono **sempre** convertiti in stringa (con jquery, con axios o con fetch), per cui il server, se necessario, dovrà eseguire il parsing.
- Eventuali parametri **vettoriali o json** devono essere preventivamente serializzati ed inviati come stringa (vedasi appunti di node.js pagina 20) per cui poi devono essere parsificati lato server.
- Esempio completo di richiesta:

```
inviaRichiesta("GET", "/api/richiesta1?id=3&opt=1&opt=2",
  {"eta": 16,  "vet":[1,2,3],  "obj":{x:5, y:7}});
```

log lato server:

```
{id: '3',  opt:['1','2'],  eta:'16',  vet:'[1,2,3]',  obj:'{"x":5,"y":7}'}
```

Notare che : id, eta ed i valori del vettore ***opt*** sono stati trasformati in stringa
Il vettore ed il json sono stati serializzati (ma i valori interni sono rimasti immutati).

Nota relative ad inviaRichiesta

- Nel caso di $.ajax() e axios i parametri passati come json all'interno del terzo parametro di inviaRichiesta, vengono salvati dentro params e quindi vengono automaticamente accodati alla URL e sovrascrivono quelli eventualmente già accodati, per cui la modalità mista non è possibile.
- Nel caso di **fetch** è compito del programmatore prelevare i parametri provenienti da inviaRichiesta ed accodarli manualmente alla URL, possibilmente senza sovrascrivere quelli già eventualmente accodati. Per cui in questo caso la modalità mista diventa possibile

Sintassi

I parametri GET vengono restituiti all'interno di **req.query.nomeParametro**

```
console.log(req.query.id, req.query.opt)
```

Accesso ai parametri POST

Per leggere i parametri POST occorre installare il modulo body-parser (nelle ultime versioni nativo come *static*)

```
npm install body-parser
let bodyParser = require('body-parser');
```

La lettura dei parametri può poi essere eseguita attraverso i seguenti middleware che restituiscono i parametri all'interno di **req.body**, come json 'pulito' senza tutte le complicanze dei parametri GET.

```
app.use(bodyParser.urlencoded({limit:'50mb', extended:true}));
app.use(bodyParser.json({limit:'50mb'}));
```

Il **primo** middleware intercetta i parametri url-encoded
Il **secondo** middleware intercetta i parametri scritti in formato json

L'opzione **`limit`** serve per l'uload di immagini base64 all'interno dei parametri post
L'opzione `extended:false` consente di passare come parametri url-encoded stringhe, vettori e anche oggetti json semplici ma non annidati
L'opzione `extended:true` consente anche l'utilizzo di oggetti estesi (json annidati)
Il default è true, però se si omette l'opzione extended il compilatore segnala un warning piuttosto fastidioso.

Il metodo delete

Secondo la prassi HTTP, soltanto i parametri strettamente necessari per la creazione o la modifica di una risorsa vanno passati nel body della richiesta. Questo vale per le richieste POST (record da inserire), PATCH (azione da eseguire), PUT (dati da sostituire).

Nel caso della DELETE, lo standard non definisce un payload con parametri, e molti server **ignorano o rifiutano il body**. Questo perché in realtà i parametri non sono strettamente obbligatori in quando l'id del record da eliminare può essere passato all'interno della risorsa.

Pertanto, l'identificatore della risorsa da eliminare viene normalmente passato nella URL, mentre eventuali parametri di filtro sono facoltativi e dovrebbero essere trasmessi in query string (URL-encoded).

Passaggio dei parametri come parte della risorsa

La tendenza attuale è quella di passare i parametri GET **non** accodandoli alla risorsa nel formato tradizionale della queryString **`?nome=valore`**, ma di inserirli direttamente all'interno della risorsa richiesta separandoli semplicemente tramite SLASH. Dovrà essere il codice server a gestire questi parametri in modo diverso, andando a leggerli direttamente all'interno della risorsa richiesta.

Supponendo di voler inviare al server la seguente richiesta GET:
`http://localhost:1337/api/risorsa?nome=pippo&eta=16`

i due parametri `nome` e `eta` possono essere direttamente accodati alla risorsa **in forma anonima ed in modo posizionale,** cioè basato esclusivamente sull'ordine di scrittura:
`http://localhost:1337/api/risorsa/`**`pippo/16`**

Il parametro passato in questo modo non sarà intercettabile all'interno della `queryString` tradizionale, ma occorrerà una apposita route scritta nel modo seguente:

```
app.get('/api/risorsa/:nome/:eta', function (req, res, next) {
    let nome = req.params.nome;
    let eta  = req.params.eta;
```

Questa tecnica vale non solo per chiamate GET, **ma anche per tutte le possibili chiamate http**.
Infatti in qualsiasi chiamata è sempre possibile passare sia parametri url-encoded (restituiti all'interno di **`req.query`**) sia all'interno del body (restituiti all'interno di **`req.body`**)

Vantaggi:
- il client può evitare l'utilizzo dei names
- E' disponibile un metodo aggiuntivo per la validazione preventiva dei parametri ricevuti

Parametri Facoltativi:Se scrivo una route del tipo **`app.use('/api/risorsa/:nome/:eta)`**,
se i prametri non ci sono la route non viene intercettata perchè si aspetta una risorsa "completa" del tipo **`/risorsa/pippo/16`**. Questo problema può essere risolto indicando gli ultimi parametri come facoltativi, facendoli seguire da un punto interrogativo: **`app.use('/api/risorsa/:nome/:eta?)`**

Cioè il parametro *`nome`* deve SEMPRE essere presente, il parametro *`eta`* diventa facoltativo.
Notare che, come nel caso delle chiamate GET, **i valori numerici vengono restituiti come stringa**

Validazione dei parametri passati come risorsa

I parametri passati come risorsa possono essere intercettati preventivamente e validati attraverso un metodo **app.param()** abbinato al metodo **app.use()** che definisce il nome dei parametri.

Nel momento in cui app.use( ) intercetta una richiesta e definisce il nome di una parametro passato come risorsa, automaticamente verifica se esiste una route di tipo **app.param**() associata a quale nome.
Se esiste la esegue **prima di procedere con la route corrente**.
Lo scopo di **app.param()** è quello di verificare la correttezza dei parametri ricevuti ed eventualmente interrompere il programma prima ancora dell'esecuzione della route effettiva identificata da app.use()

```
app.param('eta', function(req, res, next, valore) {
    console.log('\t doing name validations on ' + valore);
    if (valore!=13) next();
    else { let err = new Error('valore non consentito');  next(err); }
})
```

app.param,() può essere scritto indifferentemente prima o dopo rispetto al corrispondente app.use().

La gestione degli errori

Nel caso in cui durante l'esecuzione si verifichi un qualsiasi tipo di errore, il router automaticamente richiama uno specifico middleware caratterizzato da una firma avente non **tre** parametri (req, res, next), ma **quattro** (**err**, req, res, next). Notare che il gestore degli errori deve necessariamente avere tutti e quattro parametri; il parametro **next** NON può essere omesso.

```
app.use(function(err, req, res, next) {
      console.log("************* SERVER ERROR ***************\n", err.stack)
      res.status(500).send(err.message)
});
```

Il listener di errore dovrà essere posizionato alla fine dello script dopo tutti i listener di risposta.
Se Il listener di errore non è presente, il server si blocca **terminando** la sua esecuzione.

err.message va bene per il client ma non per il server perchè visualizza l'ultimo messaggio in cima allo stack ma non lo stack complete con la riga in cui si trova l' errore.

Il catch nelle promise

Nel caso di metodi che restituiscono una promise (es **await client.connect()**), in caso di errore la promise genera l'evento **catch**. Se il catch non è gestito (ma è gestito solo l'evento **then)**, **questo evento non si genererà mai** e di conseguenza dopo un po' l'applicazione va in timeout (e quindi in errore) **purtroppo però senza richiamare la route di errore**.
Per cui, per fare le cose bene, il catch delle promise dovrebbe ***sempre*** essere gestito.

Generazione di un errore

E' anche possibile generare nuovi errori ed interrompere l'esecuzione passando un apposito oggetto **err** al metodo next. Da usare ad esempio in caso di parametri mancanti o di valori non validi.

```
app.get('/api/data', function (req, res, next) {
    let nome = req.query.nome;
    if(!nome) {
        let err = new Error("Bad Request. Manca il parametro NOME");
        next(err);  }
    else res.send("good morning " + nome + " " + eta );
});
```

La stringa passata a **new Error** sarà quella in cima allo stack e restituita da **err.message**

I server CRUD (Create Read Update Delete)

Le operazioni base che si possono eseguire sui record di un database relazionale sono tipicamente quattro: **Create**/Insert, **Read** (Select), **Update** e **Delete** operazioni solitamente indicate con l'acronimo CRUD.

I principali metodi con cui un client http può accedere ad una risorsa sono essenzialmente anche quattro: GET, POST, PATCH/PUT e DELETE.

Un server CRUD è un http server che, facendo riferimento sempre alla medesima risorsa, associa i quattro metodi HTTP con le quattro operazioni CRUD. Cioè il medesimo servizio si comporta in modo diverso a seconda del metodo con cui viene effettuata la chiamata:

GET -> Lettura di un record
POST -> Inserimento di un nuovo record passato come parametro
DELETE -> Cancellazione di un record il cui ID di solito viene passato come risorsa */people/13*
PATCH -> Aggiornamento di alcuni campi di un record il cui ID viene passato come risorsa */people/13*
PUT -> Completa sostituzione di un record il cui ID viene passato come risorsa */people/13*

Notare che questo principio è in contrasto con quello che è l'utilizzo comune dei metodi HTTP.
Molto spesso infatti il metodo GET viene utilizzato per eseguire qualsiasi tipo di interazione con il server.
Ad esempio, in una applicazione Web, per l'inserimento di un nuovo record si può comunemente eseguire una richiesta di tipo GET su un URI del tipo:

```
http://www.myapp.com/addCustomer?name=Rossi
```

Questo utilizzo del metodo GET ad ampio spettro non è però conforme ai principi **REST**, secondo i quali il metodo GET dovrebbe servire **esclusivamente** per accedere ai dati di una risorsa.

Richiami sugli HTTP Request Method

Oltre ai tre metodi base (GET, POST e HEAD), in HTTP 1.1 sono stati aggiunti altri **sei Request Method** che consentono di specificare meglio l'azione che deve essere eseguita su una certa risorsa. Tranne POST (e in parte PATCH) tutti i metodi sono di tipo **`idempotent`**, cioè se **la stessa richiesta** viene inviata più volte produce sempre lo stesso risultato senza modificare lo stato del server (che è uno dei pilastri di HTTP). Cioè se il metodo viene richiamato una sola o più volte consecutive, il risultato è sempre lo stesso. Aspetto molto importante nell'ambito della fault-tolerance. Se un comando idempotent va in timeout (perché si perde la risposta del server) lo posso rieseguire una seconda volta senza alcun rischio.

POST non è idempotent perché ad ogni richiesta aggiunge un nuovo record facendo variare lo stato del server. DELETE è idempotent perché alla prima esecuzione cancella il record, mentre nelle esecuzioni successive non esegue alcune azione, per cui il risultato finale è indipendente dal numero di richiami.

`GET`	The `GET` method requests a representation of the specified resource. **Requests using `GET` should only retrieve data**.
`HEAD`	The `HEAD` method asks for a response identical to that of a `GET` request, but without the response body.
`POST`	**The `POST` method is used to submit an entity to the specified resource**, often causing a change in state or side effects on the server.
`PUT`	The `PUT` method **replaces all current representations of the target resource with the request payload**. // Sostituisce l'intera risorsa oppure crea una nuova risorsa nel caso in cui il record ricercato non esista (es opzione **upsert** di mongodb)
`PATCH`	The `PATCH` method is used to **apply partial modifications to a resource**.
`DELETE`	The `DELETE` method **deletes the specified resource**.
`OPTIONS`	The `OPTIONS` method is used to describe the communication options for the target resource.
`TRACE`	The `TRACE` method performs a message loop-back test along the path to the target resource.
`CONNECT`	The `CONNECT` method establishes a tunnel to the server identified by the target resource.

Accesso da smartphone e Hotspot mobile

Se PC Server e smartphone appartengono alla stessa rete locale, lo smartphone può accedere al server semplicemente impostando nella barra di navigazione l'indirizzo ip locale del server.
Ad esempio `http:// 192.168.1.2:1337` oppure `https:// 192.168.1.2:1338`

Se invece PC Server e smartphone **non** appartengono alla stessa rete locale, Windows 10 fornisce un comodissimo servizio di **Hotspot mobile** che consente di trasformare il modulo WiFi del PC in un Access Point / DHCP server verso l'esterno. Lo smartphone potrà così collegarsi alla rete dell'hotspot mobile.

Infine per i PC ***non dotati di moduli WiFi,*** si può montare sul PC una antennina USB esterna che sostanzialmente implementa il modulo WiFi.

Configurazione / attivazione dell'Hotspot mobile

- Start / **Impostazioni** / Rete e Internet / **Hotspot mobile** (menù di sinistra)
- Condividi la connessione Internet con altri dispositivi / **Attiva**
- Eventualmente Modificare / Nome Rete e Password

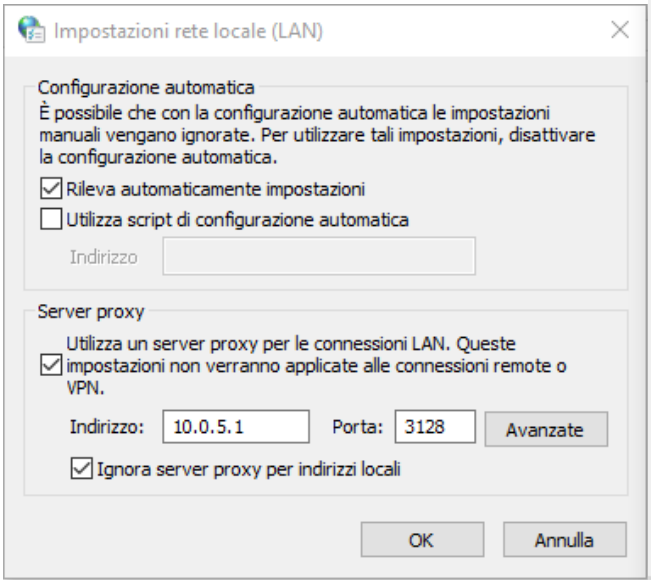
L'hotspot si crea lui una nuova rete (typ **192.168.137.0**) riservando il primo indirizzo per se stesso (**192.168.137.1**) ed assegnando gli altri indirizzi via DHCP ai dispositivi che ne fanno richiesta.

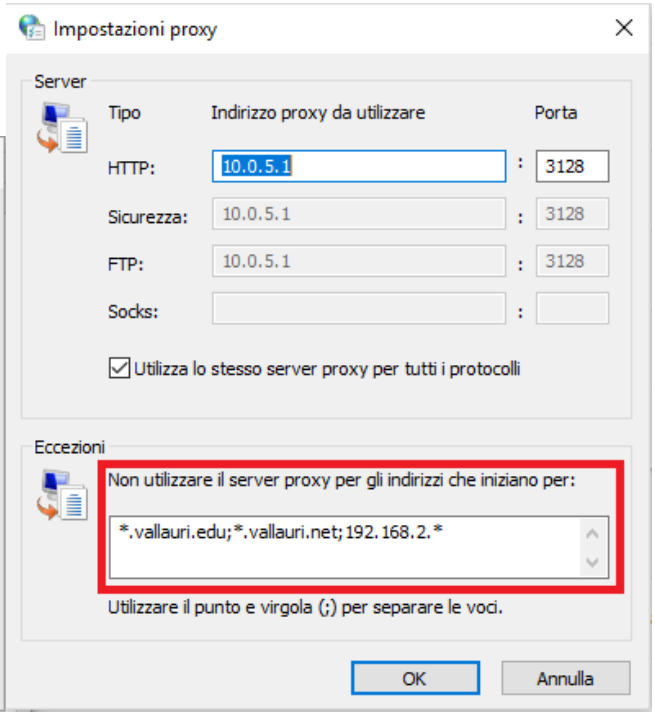
Dopo di che per accedere ad un server in esecuzione sul PC occorre:
- avviare il server in ascolto sull'indirizzo **192.168.137.1** (oppure senza specificare nessun indirizzo IP, quindi su tutte le interfacce)
- Aprire il browser sul dispositivo mobile e puntare all'indirizzo **192.168.137.1:1337**

E' consentito un max di **8 device** contemporanei

Impostazione / Eliminazione del Proxy da Pannello di Controllo

Pannello di controllo
Centro connessioni rete
Opzioni Internet (in basso a sinistra)
Connessioni / Impostazioni LAN





Gestione delle richieste extra domain (CORS)

Cross-Origin Resource Sharing

Per ragioni di sicurezza, storicamente i ***browser*** non potevano inviare richieste **`extra-domain`** (**`cross-origin request`**), cioè richieste indirizzate verso ***origini*** diverse rispetto all'origine stessa da cui era stata scaricata la pagina, dove per ***origine*** si intende l'insieme delle seguenti informazioni:

- **protocollo** utilizzato per l'accesso alla pagina (http, https)
- **dominio** da cui la pagina è stata scaricata (es localhost)
- **porta** di ascolto del server da cui la pagina è stata scaricata.

Ad esempio **`http://localhost:3000`**

Evoluzione storica

Il fatto che per lungo tempo i browser non potessero in nessun modo inviare richieste extra-domain comportava massima sicurezza ma scarsissima flessibilità. I programmatori hanno iniziato allora a realizzare dei trucchi per aggirare il problema. Ad esempio:

- esecuzione di un submit con target="`iframe`" (i submit non sono sottoposti alle limitazioni CORS)
- utilizzo di un tag**`<script>`** (che anch'esso non è soggetto alle limitazioni CORS) che accede sotanzialmente ad un servizio lato server il quale espone dei dati mediante un formato particolare denominato JSONP (JSON with padding)

Ad un certo punto, dopo lunghe discussioni, gli enti di standardizzazione hanno deciso di acconsentire al fatto che i browser potessero inviare richieste extra domain, con l'introduzione però di specifiche validazioni lato server, le cosiddette limitazioni CORS.

CORS (Cross-Origin Resource Sharing) è il meccanismo utilizzato lato server per decidere se accettare o meno le richieste extra-domain provenienti da un browser.

In caso di richiesta ritenuta non valida, il server risponderà con un errore del tipo "**`Blocked for CORS policy error`**" che il browser provvederà a visualizzare.

Nota: Occorre precisare che le limitazioni CORS valgono soltanto per le richiete provenienti dai **browser**. Non valgono invece per richieste **server to server** o per richieste provenieti da una **app nativa**.
Sono invece sottoposte a vincoli CORS le **app ibride** (buildate con cordova o capacitor), questo perchè una applicazione ibrida in realtà è costituita da pagine html visualizzate all'interno del webViewer (browser) locale il quale le richiede ad un web server locale (il cui nome può essere impostato a piacimento, ad esempio in **cordovaApp).** Quindi le richieste ajax provengono da un **browser**.

SAFE REQUEST e UNSAFE REQUEST

Sono considerate UNSAFE tutte le richieste che possono alterare il contenuto del database:

safe request: GET, HEAD (che è una get alla quale il server risponde con le sole intestazioni),
OPTIONS (che serve per verificare eventuali permessi)
unsafe request: PUT, DELETE, POST, PATCH

NOTA: SAFE implica idempotenza ma non viceversa, dove per idempotenza si intende una proprietà per la quale applicando molteplici volte una data funzione, il risultato ottenuto è uguale a quello derivante dall'applicazione della funzione un'unica volta. All safe methods are also idempotent, but not all idempotent methods are safe. For example, PUT and DELETE are both idempotent but unsafe

Invio di una richiesta HTTP

Nel momento in cui il browser invia una HTTP REQUEST ad un server, automaticamente **aggiunge fra le intestazioni HTTP un apposite campo `Origin`** indicante l'origine di provenienza della richiesta.
Ad esempio:

```
GET /api
Host: randomuser.me
Origin: http://localhost:1337     // protocollo, dominio e porta del richiedente
```

Assenza di specifiche CORS

In assenza di specifiche CORS, il server accetta SOLO ed ESCLUSIVAMENTE le richieste provenienti dalle pagine appartenenti allo stesso dominio.
Queste pagine possono comunque inviare richieste HTTP SAFE e UNSAFE di ogni genere (get, post, put, patch, delete). La HTTP response conterrà una intestazione speciale indicando l'origne ammessa:

```
Access-Control-Allow-Origin = "http://localhost:1337"
```

Questa intestazione può contenere anche un elenco di origini valide separate da virgola.

In alternative il server potrebbe gestire l'intestazione nel seguente modo, indicando cioè che tutte le origini sono ammesse:

```
Access-Control-Allow-Origin = "*"
```

Presenza di specifiche CORS

In preseza di specifiche CORS, il server, ricevuta la richiesta, ispeziona il campo `Origin` e decidere se accettare o meno la richiesta sulla base di una apposite **`white list`**.

- Se l'origine è consentita, confeziona la risposta e aggiunge l'apposita intestazione **`Access-Control-Allow-Origin`**.
- Se l'origine non è consentita risponde con un errore CORS

Il server può anche aggiungere una intestazione **`Access-Control-Max-Age = 86400`** (1 day) la quale fa si che il browser memorizzi in cache che la destinazione è valida e quindi, nelle richieste successive, non inserisca più il campo **`Origin`** (fino alla scadenza del timeout) velocizzando le operazioni.
Nota: se il browser non inserisce l'intestazione **`Origin`** il server di default accetterà la richiesta.

Il browser funge da mediatore: ricevuta la risposta dal server controlla l'intestazione `Access-Control-Allow-Origin.` In caso di origine valida il browser restituisce i dati alla callback javascript, altrimenti visualizza un CORS Policy Error.

La libreria *express CORS*, anche in caso di richieste intra-domain, accetta come valide soltanto le **SAFE REQUEST,** cioè in pratica SOLTANTO le richieste GET.
Tutte le altre richieste, in assenza di apposita entry nella white list, generano un errore CORS.
Per cui all'interno della white list occorre inserire sempre anche l'indirizzo stesso del server.

Accesso alle intestazioni

In caso di richiesta accettata, java script potrà accedere soltanto ad alcune **HTTP HEADERS**, le cosiddette **SAFE HEADERS** considerate innocue dal punto di vista della sicurezza

```
Content-Language    Content-Length    Content-Type      Last-Modified
Cache-Control        Expires      Pragma
```

Il tentativo di accesso a qualunque altra intestazione genera errore.

Per consentire a javascript di accedere anche ad altre intestazioni, il server deve indicare esplicitamente all'interno di **Access-Control-Expose-Headers** l'elenco delle intestazioni accessibili. Ad es per consentire l'accesso all'header **authorization** il server dovrà impostare la seguente intestazione:

Access-Control-Expose-Headers = "authorization"

Esempi di codice lato server

Soluzione 1

Settare manualmente tutte le intestazioni. Abbastanza laborioso

```
app.use("/", function(req, res, next) {
    res.setHeader("Access-Control-Allow-Origin", "http://localhost:1337")
    res.setHeader("Access-Control-Allow-Methods", "GET,POST,PATCH,DELETE");
    res.setHeader("Access-Control-Allow-Headers", "*");
    res.setHeader('Access-Control-Allow-Credentials', true);
    next();
```

Soluzione 2

Utilizzare un'apposite libreria **cors** che, sulla base di una **whitelist,** va lei a settare opportunamente tutte le intestazioni necessarie. La **whitelist** è semplicemente un vettore enumerativo contenente un elenco di **origini** abilitate ad inviare richieste. Se la pagina che sta inviando la richiesta appartiene ad una di queste origini, il server accetterà la richiesta (qualunque sia il metodo di chiamata) e setterà appositamente le intestazioni.

```
const cors = require('cors')

const whitelist = [
     "http://my-crud-server.herokuapp.com ",     // porta  80 (default)
     "https://my-crud-server.herokuapp.com ",    // porta 443 (default)
     "http://localhost:3000",
     "https://localhost:3001",
     "http://localhost:4200",                    // server angular
     "https://cordovaapp",                       // porta 443 (default)
];

const corsOptions = {
    origin: function(origin, callback) {
        if (!origin)      // browser direct call
            return callback(null, true);
        if (whitelist.indexOf(origin) === -1) {
            let msg = `The CORS policy for this site does not
                                allow access from the specified Origin.`
            return callback(new Error(msg), false);
        }
        else
            return callback(null, true);
    },
    credentials: true
};
app.use("/", cors(corsOptions));
```

NB : **Se si utilizza questa libreria,** occorre indicare all'interno della **white list** anche la URL del server medesimo. Infatti, come detto, per default anche in caso di richieste intra-domain la libreria CORS accetta come valide **soltanto le SAFE REQUEST,** cioè in pratica SOLTANTO le richieste GET.

Soluzione 3

Usare sempre la libreria precedente però 'aprendo' a qualsiasi **origin** :

```
const corsOptions = {
    origin: function(origin, callback) {
          return callback(null, true);
    },
    credentials: true
};
app.use("/", cors(corsOptions));
```

Ad esempio le API di **randomuser.me** possono essere richiamate da qualsiasi dominio però solo in **https**
Viceversa molte altre API non forniscono servizio CORS e, in questi casi non si può fare nulla.

- Se si invia la richiesta dalla barra di navigazione del browser tutto ok.
- Se invece si invia una richiesta Ajax viene fuori un errore CORS.

Concetto di Preflight Request

Una richiesta è considerate **SIMPLE** se soddisfa due condizioni:

- Utilizza un SIMPLE method (GET, POST, HEAD)
- Utilizza soltanto delle **SAFE REQUEST HEADERS**, che sono Quattro in tutto:

  `Accept, Accept-Language, Content-Language, Content-Type`

  Accept, usato nelle request, indica indica quali tipi di contenuto, espressi in MIME Type, il client è in grado di comprendere. Esempio: `Accept: text/html, image/*.` Il server seleziona una delle proposte e informa il client della scelta con l'intestazione di risposta **Content-Type**.
  Content-Type, nelle request indica il tipo dei parametri (sempre come MIME Type).
  Nelle response invece indica il tipo della risposta
  Accept-Language, usato nelle request indica indica la lingua naturale preferita dal client (che nelle pagine html è indicata all'interno dell'attributo `<head lang="it">`)
  Content- Language, usato nelle response indica la lingua naturale della response stessa

Le richieste che non soddisfano a questi criteri non sono considerate SIMPLE.
Inoltre l'intestazione **Content-Type** può assumere solo tre valori:
`application/x-www-form-urlencoded, multipart/form-data. text/plain.`

Non è pertanto ammesso il valore **application/json**. Dunque una chiamata POST, pur essendo intrinsecamente SIMPLE, se passa nel body dei parametri JSON diventa automaticamente non SIMPLE.

Quindi in pratica le richieste SIMPLE sono assimilabili alle richieste SAFE con le seguenti aggiunte:

- **Le chimante POST con parametri url-encoded e con SAFE REQUEST HEADERS sono considerate SIMPLE**
- **Le chimante GET contenenti UNSAFE REQUEST HEADERS sono considerate NON SIMPLE**

Le **SIMPLE Request** vengono gestite come indicato nei paragrafi precedenti.

NON SIMPLE Request e concetto di PLEFLIGHT

Nel caso delle sole richieste extra-domain di tipo NON SIMPLE il browser non esegue direttamente la request ma prima invia una richiesta, chiamata "preflight request", per richiedere il permesso al server. Una preflight request utilizza il metodo OPTIONS, nessun body e due headers:

Access-Control-Request-Method che indica il method della successiva request.
Access-Control-Request-Headers che contiene una lista degli **UNSAFE REQUEST Headers** (separati da virgola) che saranno inseriti nella successiva request.

Esempio di preflight request:

```
OPTIONS /updateRecord
Host: anywhere.com
Origin: http://localhost:1337
Access-Control-Request-Method: PATCH
Access-Control-Request-Headers: Content-Type, API-Key
```

Se il server accetta di servire la request, invia una risposta con uno status 200, body vuoto, e le seguenti headers (oltre alla `Access-Control-Allow-Origin` sempre presente):

`Access-Control-Allow-Methods` con i metodi autorizzati, separati da virgola.
`Access-Control-Allow-Headers` con la lista delle headers autorizzate.

Inoltre, l'header **Access-Control-Max-Age** può specificare il n di secondi di cache dei permessi. In questo modo il browser non necessita di ulteriori invii preflight per le seguenti requests coerenti con i permessi concessi

Esempio di preflight response:

```
200 OK
Access-Control-Allow-Origin: http://localhost:1337
Access-Control-Allow-Methods: PUT,PATCH,DELETE
Access-Control-Allow-Headers: API-Key, Content-Type, Cache-Control
Access-Control-Max-Age: 86400
```

Il browser quindi invia la **main request** e provvede a memorizzare nella cache la preflight response per il tempo indicato, (86400 secondi, un giorno), in modo che le richieste successive non richiedano preflight. Se le successive request si adattano alle autorizzazioni in cache, potranno essere inviate direttamente.

La metodologia per l'invio della main request è la stessa utilizzata per le simple request.
Il browser potrà inserire soltanto le HEADERS autorizzate dalla precedente response.
Esempio di main request relativa alla preflight request precedente:

```
PATCH //updateRecord
Host: anywhere.com
Origin: http://localhost:1337
Content-Type: application/json
API-Key: secret
```

Infine il server, nella risposta, deve ripetere l'header allow-origin:

```
Access-Control-Allow-Origin: "http://localhost:1337"
```

Cookies e Credenziali

Una cross-origin request per impostazione predefinita non fornisce credenziali (cookie o autenticazione HTTP), nemmeno i cookie appartenenti al dominio di destinazione. Nella preflight il browser può richiedere al server l'autorizzazione all'invio dei cookies (nel caso di fetch : `credentials: "include")`.
Il server nella response dovrà aggiungere la seguente intestazione che abilita il browser alla trasmissione delle credenziali e dei cookies:

```
res.setHeader('Access-Control-Allow-Credentials', true);
```

Notare che se simposta questa header, `Access-Control-Allow-Origin` non può essere assumere il valore * , ma deve contenere esattamente l'origine del richiedente.

Creazione di un server HTTPs

Il protocollo HTTPs (basato su TLS/SSL) crea un canale cifrato tra client e server. Prevede un tipo di autenticazione detta ***unilaterale*** in cui solo il server è autenticato (cioè il client conosce l'identità del server), ma non viceversa (il client rimane anonimo e non autenticato). Il client valida il certificato del server controllando la firma digitale e verificando che questa sia valida e riconosciuta da una *Certificate Authority*.

Il passo necessario per poter creare un server https è quello di creare una **coppia di chiavi RSA**:

- una **chiave privata** da tenere memorizzata sul server
- una **chiave pubblica** da distribuire tramite **Certificato Digitale** controfirmato digitalmente da una Certification Authority. Il Certificato Digitale contiene la chiave pubblica più altre informazioni sul dominio e sull'ente a cui la chiave è associata. Sostanzilamente certifica che quella chiave appartiene a quel dominio.

I formati di memorizzazione di chiavi e certificati

.pem indica una chiave RSA memorizzata in formato testuale base64
.der indica una chiave RSA memorizzata in formato binario
.key estensione generica per una chiave RSA, tipicamente memorizzata in formato pem

.crt estensione di un Certificto Digitale **X.509**, memorizzato in formato testuale base64
.csr o **.req** indicano un `Certificate Signing Request` cioè un Certificato Digitale da inviare ad una Certification Authority per l'apposizione della firma digitale.

Creazione delle chiavi RSA

RSA Key Generator

Crea una coppia di chiavi RSA (*typ 2048 bit***)**. Restiutuisce direttamente la chiave pubblica in formato .pem **senza però creare il Certificato**. I comandi che usa sono i seguenti:

1. genera una chiave privata da salvare come private.pem
```
openssl genrsa
```
2. genera la corrispondente chiave pubblica
```
openssl rsa   -in private.pem   -pubout   -out public.pem
```
3. le chiavi possono poi essere usae per cifrare / decifrare un file data.txt
```
openssl rsautl -encrypt -inkey public.pem -pubin -in data.txt -out data.txt.enc
openssl rsautl -decrypt -inkey private.pem -in data.txt.enc -out data.txt
```

https://www.devglan.com/online-tools/generate-self-signed-cert

Crea una coppia di chiavi RSA in cui la chiave pubblica viene restituita sotto forma di Certificato. Richiede pertanto tutte le varie informazioni da inserire nel certificato e crea:

- chiave privata
- certificato X.509 self-signed
- certificato.cer da inviare ad una Certification Authority per la firma digitale

Per visualizzare il contenuto del Certificato X.509 è sufficiente fare doppio click sul certificato stesso

letsencrypt.org

Consente di creare un Certificato controfirmato da una Certification Authority che però deve necessariamente essere collegato ad un dominio esistente. La procedura da utilizzare è molto semplice, specialmente se si utilizza il cosiddetto *manual mode*

OpenSSL

OpenSSL è una applicazione che consente di creare certificati self-signed ma anche controfirmati.
Il sito ufficiale ***openssl.org*** fornisce e mantiene soltanto la versione Linux.
Al momento (novembre 2018) l'ultima versione è la v ***1.1.1d*** .

Per quanto riguarda le versioni windows si consigliano i seguenti link:

https://slproweb.com/products/Win32OpenSSL.html (con eseguibile di installazione)
sourceforge.net che fornisce una più flessibile versione .zip che può essere 'copiato' in qualsiasi cartella

Per ogni versione sono disponibili due distribuzioni:

- quella ***full*** (circa 43MB) raccomandata per gli sviluppatori (*indispensabile per la creazione dei certificati*) e che necessita di un file di configurazione distribuito insieme al pacchetto
- quella ***light*** (circa 3MB) raccomandata per i semplici utilizzatori e che non necessita del file di configurazione. Ad esempio la libreria di node OPCUA client per il funzionamento richiede la presenza di openssl nel profilo utente, ed è *sufficiente* la versione light

openssl in realtà viene installato insieme a *git*, per cui se si ha git installato sul pc non c'è bisogno di ulteriori installazioni.

- Nelle versioni più vecchie occorreva creare manualmente le seguenti variabili di ambiente:
  PATH -> C:\programmi\git\usr\bin
  OPENSSL_CONF -> C:\programmi\git\usr\ssl\openssl.cnf
  (***OPENSSL_CONF*** esiste solo nella versione full)

- Nelle versioni più recenti queste variabili di ambiente sono create automaticamente da git in fase di installazione

Per creare le chiavi il comando da utilzzare è il seguente:

openssl req -x509 -days 365 -newkey rsa:2048 -nodes -keyout privateKey.pem -out certificate.crt

Se non si impostano le variabili di ambiente si può lanciare il comando direttamente dalla cartella BIN di openssl. E' anche possibile digitando solo openssl aprire un terminale all'interno del quale si possono inviare gli stessi comandi omettendo la voce iniziale openssl.

Appena lanciato, il comando precedente chiede all'utente di inserire alcune informazioni da inserire all'interno del Certificato:

We will be asked a few questions about our server in order to embed the information correctly in the certificate.
Fill out the prompts appropriately. The most important line is the one that requests the **Common Name** (server FQDN Fully Qualified Domain Name). You need to enter **the domain name associated with your server** or your server's public IP address.

```
Output
Country Name (2 letter code) [AU]:US
State or Province Name (full name) [Some-State]:New York
Locality Name (eg, city) []:New York City
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Bouncy Castles, Inc.
Organizational Unit Name (eg, section) []:Ministry of Water Slides
Common Name (e.g. server FQDN or YOUR name) []:server_IP_address
Email Address []:admin@your_domain.com
```

Significato dei parametri

https://www.digitalocean.com/community/tutorials/how-to-create-a-self-signed-ssl-certificate-for-apache-in-ubuntu-16-04

- **req**: This **subcommand** specifies that we want to use X.509 certificate signing request (CSR) management. The "X.509" is a public key infrastructure standard that SSL and TLS adheres to for its key and certificate management. We want to create a new X.509 cert, so we are using this subcommand.
- **-x509**: This further modifies the previous subcommand by telling the utility that *we want to make a self-signed certificate* instead of generating a certificate signing request, as would normally happen.
- **-days 365**: This option sets the length of time that the certificate will be considered valid. We set it for one year
- **-newkey rsa:2048**: This specifies that we want to generate a new certificate and **a new private key** at the same time. We did not create the key that is required to sign the certificate in a previous step, so we need to create it along with the certificate. The `rsa:2048` portion tells it to make an RSA key that is 2048 bits long.
- **-nodes**: This tells OpenSSL **to skip the option to secure our certificate with a passphrase**.
  We need Server to be able to read the file, without user intervention, when the server starts up.
  A passphrase would prevent this from happening because we would have to enter it after every restart.
- **-keyout**: This line tells OpenSSL where to place the generated private key file that we are creating.
- **-out**: This tells OpenSSL where to place the certificate that we are creating.

Creazione di un server HTTPs basato su express

```
const privateKey  = fs.readFileSync("keys/privateKey.pem", "utf8");
const certificate = fs.readFileSync("keys/certificate.crt", "utf8");
const credentials = { "key": privateKey, "cert": certificate };

let express = require('express');
let app = express();

let httpServer = http.createServer(app);
httpServer.listen(8080);

let httpsServer = https.createServer(credentials, app);
httpsServer.listen(HTTPS_PORT, function(){
    console.log("Server in ascolto sulle porte HTTP:" + PORT + ", HTTPS:" + HTTPS_PORT)
});
```

I due server http e https possono essere in esecuzione contemporanea consentendo accesso http e https
Quando si usa https ricordarsi di aprire le **CORS Policy** ad https e aggiungere **`secure=true`** al cookie.

Upload e storage di un file binario

Lato Client

Il controllo **`input[type=file]`**, in corrispondenza dell'evento **onchange** (cioè dopo aver selezionato un file), restituisce all'interno dell'attributo dinamico **`files`** un **vettore enumerativo di `Blob`** di tipo File, in cui ogni Blob contiene, tra le altre cose, l'immagine binaria da trasmettere.

Per trasmette una immagine binaria al server, occorre caricarla all'interno di un oggetto denominato **FormData** che accetta al suo interno qualunque tipo di dato.

```
let formData = new FormData()
formData.append('nome', txtNome.value);                // string
formData.append('maggiorenne', chkMaggiorenne.checked // boolean
formData.append("myJSON", JSON.stringify(myJSON))     // json
formData.append("txtFile1",  txtFiles.files[0]);      // blob
```

- **Tutti i parametri 'normali' (cioè diversi dai Blob) vengono automaticamente convertiti in stringa**
- All'intero di FormData è anche possibile passare un intero json, che però **deve essere serializzato** manualmente, altrimenti viene applicato il metodo .toString() che restituisce Obiect Obiect

Trattandosi di un upload, occorre eseguire una richiesta **POST** passando come parametro l'oggetto **`formData`** come flusso binario senza aggiungere controlli / serializzazioni.

```
let request = inviaRichiesta("POST", "/api/upload", formData);
```

Lato Server

Per intercettare il file lato server è disponibile la libreria **`express-fileupload`** che intercetta :

- sia i **Blob** che restituisce all'interno di **`req.files`**
- sia gli eventuali parametri scalari che restituisce all'interno di **`req.body`**

```
const fileupload = require('express-fileupload');

app.use(fileupload({
    "limits": { "fileSize": (20 * 1024 * 1024) }  // 20 MB
}));
```

All'interno di **`req.files`** viene restituito :

- **in caso di file singolo un oggetto UploadedFile**
- **in caso di files multipli un vettore enumerativo di UploadedFile**

*Nota: A differenza di **php**, in cui di solito i parameri post vengono passati in formato url-encoded, le parentesi quadre dopo il nome della variabile client da trasmettere al server sono del tutto irrilevanti.*

Struttura di un uploadedFile

UploadedFile è un json in cui vengono riportati alcuni campi del Blob ricevuto dal client, ma non tutti.
I campi ritornati sono i seguenti:

`.name` // Il filename impostato dal client (privo del path)
`.size`
`.mimetype`
`.data`: Un oggetto **Buffer** contenente lo stream binario vero e proprio
`.truncated`: A boolean that represents if the file is over the sizeLimit
`.mv`: A function to move the file elsewhere on your server
`.md5`: A function that returns an MD5 checksum of the uploaded file

il metodo **myFile.mv(path)** è quello che consente la memorizzazione fisica del file.

```
app.post('/api/upload', function (req, res, next) {
    if (!req.files || Object.keys(req.files).length == 0)
           res.status(400).send('No files were uploaded');
    else{
        let file = req.files.txtFile;
        file.mv('./static/upload/' + file.name, function(err) {
                if (err) res.status(500).send(err.message);
                else res.json("ok");
       })
    }
```

In alternativa si potrebbe anche utilizzare direttamente `fs.writeFile`

```
    let imgPath = "./static/upload/" + file.name
    fs.writeFile(imgPath, file.data, function(err){} );
```

In entrambi i casi se il **file** esiste già viene semplicemente sovrascritto.

Viceversa la **cartella** in cui vengono salvati i files (nell'esempio **upload**) **deve già esistere**, altrimenti il metodo **file.mv()** va in errore.

Eventualmente si potrebbe aggiungere il seguente codice che va a verificare l'esistenza della cartella e, qualora la cartella non sia presente, la crea:

Sia **path** il path relativo dell'immagine da salvare (ad esempio `path= "img/miaImmagine.jpg"`)

```
let folderPath = path.split('/');
folderPath.pop();  // elimino il nome del file
// nel caso di più sottocartelle ricostruisco il path
folderPath = folderPath.join('/');
if (!fs.existsSync("./static" + folderPath))
     fs.mkdirSync("./static" + folderPath);
```

Il middleware multer

```
import Multer from 'multer';
const multer = Multer({ dest: 'uploads/' });
```

multer è un middleware che rappresenta una interessante alternativa a **express-fileupload**
Si si usa multer occorre non utilizzare **express-fileupload** che esegue una pre-elaborazione rendendo il file non fruibile da multer.

Se il file uploadato è uno solo, il middleware multer può essere richiamato direttamente nel servizio

```
app.post('/api/newPost', multer.single('image'), async (req, res) => {
  const { post } = req.body;
  const img:any = req.file.filename
```

image è la chiave utilizzata per il caricamento del blob all'interno di formData

multer.single('image') provvede a salvare l'immagine binaria su disco all'interna della cartella **uploads** indicata in fase di configurazione. Al file viene assegnato un nome casuale lungo 33 caratteri e privo di estensione

req.file.filename contiene il nome casuale assegnato al file e che a sua volta potrà poi essere salvato all'interno del databse

Esempio completo File Multipli

Nel caso di file multipli, dal momento che:

- **`file.mv()`** è un metodo asincrono,
- la risposta da inviare al client deve essere unica e riassuntiva

diventa abbastanza complicato gestire gli errori che possono comunque sempre verificarsi nel caso di upload di un file. Anche la gestione di un contatore è estermamente complicata perché la risposta dovrebbe essere inviata al client soltanto nel momento in cui l'ultimo file è stato salvato correttamente.

In questo caso si può utilizzare la libreria **`async`** riportata sugli appunti di nodejs (pag 29):

```
app.post('/api/upload', function (req, res, next) {
     if (!req.files || Object.keys(req.files).length == 0)
          res.status(400).send('No files were uploaded');
     else
     {
          let files = req.files.txtFile;
          async.forEach(files,
               function(file, callback ){
                    file.mv('./static/upload/' + file.name,
                         function(err) {
                            callback(err)
                         })
               },
               function(err) {
                    res.set("content-type", "application/json;charset=utf-8")
                    if (err)
                         res.status(500).json(err.message);
                    else
                         res.json("ok");
               }
          );
     }
});
```

Salvataggio su disco di un file trasmesso in base64

La trasmissione di files base64 all'interno dei parametri POST va bene per trasmettere piccoli files che devono essere salvati come stringa all'intero del database.
Per cui prima di trasmettere il file bisognerebbe per lo meno eseguire un **Resize** proporzionato.

Volendo, lato server, è comunque possibile riconvertire il file base64 in binario e salvarlo su disco

```
let imgBase64 = req.body.img;
let imgName = req.body.imgName;

let imgData = imgBase64.replace(/^data:image\/\w+; base64,/, "");
let buffer = Buffer.from(imgData, "base64");

fs.writeFile("./static/img/" + imgName, buffer, (err: any) => {
      if (err) {
            res.status(500);
            res.send(err.message);
      }
      else {
            res.status(200);
            res.send({"ris":"ok"});
      }
});
```

In pratica prima di salvare su disco si elimina l'intestazione base64 dal file, lo si trasforma in un **buffer binario** ed infine lo si scrive su disco tramite il metodo **`fs.writeFile`**

Richieste server to server

Quando una applicazione deve contattare un **server** esterno (ad esempio google maps, oppure cloudinary, oppure google AI), la soluzone migliore è sempre quella di gestire la chiamate lato server. In questo modo la chiavi di accesso al servizo rimangono confinate sul server e non possono intercettate all'interno di una pagina javascript. **Il client richiede i dati al proprio server, il quale invia una richiesta HTTP al server esterno e poi ritorna il risultato al client**

Per eseguire richieste server to server di solito sono disponibili delle API importabili direttamente all'interno del progetto.

Le chiamae server to server possono però essere utilizzate anche nel caso di API pubbliche non forniscano servizi CORS al browser. **Il server infatti NON è sottoposto a vincoli CORS..**
Per inviare una richiesta server to server si può utlizzare una qualunque delle ibrerie utilizzate lato cliet, cioè $.ajax, **axios.js** oppure fetch.

In passato si utilizzava un oggetto **request** deprecato a febbraio 2020 per ragioni di sicurezza

```
const request = require('request');
request("https://api.xxxxx.com", function(error, responseStatus, data){
    if(!error) res.send(JSON.stringify(data))
});
```

axios.js

L'utilizzo è lo stesso del lato client. Però lato server occorre aggiungere un'intestazione **Accept-Encoding** perché il default è **gzip** e quiandi la risposta arriverebbe in forma compressa:

```
const axios = require('axios');

app.get('/api/people', function(req, res, next) {
      let url = "https://randomuser.me/api?results=5"
      axios.get(url, {headers: {'Accept-Encoding': 'application/json'}})
      .then(response => {
            res.send((response.data)
      })
      .catch(err => {
            res.status(500)
            res.send("Errore accesso server esterno " + err.message)
      })
})
```

fetch

si può utilizzare lato server la stessa libreria usata lato client che continua a funzionare perfettamente

Invio di una mail tramite nodemailer

Per poter inviare mail occorre innanzitutto abilitare il proprio ***mail server*** all'invio di mail ricevute da terze parti. **nodemailer** supporta

- sia l'autenticazione semplice di tipo **Auth** basata sull'invio di username e password
- sia l'autenticazione più complessa e sicura **OAuth2** basata sull'invio di un token

Nel caso di Gmail per abilitare l'autenticazione semplice occorre andare su:
impostazioni / Account / Impostazioni account Google
Sicurezza / scorrere nella pagina e andare alla sezione **Accesso App meno sicure**

Abilitare il servizio. Questa opzione avrebbe dovuto essere disabilitata a partire dal 30 maggio 2022.
In realtà è stata prorogata fino all'autunno 2024 probabilmente per dare modo a tutti di adeguarsi.
Dopo tale data occorrerà necessariamente utilizzare una autenticazione **OAuth 2.0**

```
import nodemailer from "nodemailer";
const auth = {
      "user" : process.env.gmailUser,
      "pass" : process.env.gmailPassword,
}
const transporter = nodemailer.createTransport({
    "service": "gmail",
    "auth": auth
});



app.post('/api/newMail', function(req, res, next) {
      let mailOptions = {
            "from": environment.mailServer.user,
            "to": req.body.to,
            "subject": req.body.subject,
            // "text": req.body.data,  // contenuto testuale
               "html": req.body.data,  // contenuto html
            "attachments": [
                  {"filename":"qrcode.png",   "path":"./qrcode.png" }
            ]
      }
      transporter.sendMail(mailOptions, function (err, info) {
            if (err) {
                  res.status(500).send("Errore invio mail\n" + err.message);
            }
            else {
                  console.log(info);
                  transporter.close();
                  res.send({ris:"ok"});
            }
      })
});
```

Gestione delle Variabili SESSION

Si usa il middleware **`session`** che genera un nuovo sessionID e lo memorizza all'interno di un cookie.

```
let session = require('express-session');
app.use(session({
    secret: "myKeyword",
    name: "sessionId",
    // proprietà legate allo store
    resave: false,
    saveUninitialized: false,
    cookie: {
        secure: false,      // true per accessi https
        maxAge: 60000       // durata in msec
    }
```

Il primo parametro (secret) rappresenta la chiave da utilizzare per l'autenticazione HMAC, chiave che verrà utilizzata come chiave di hash per la cifratura irreversibile di sessionID
Il secondo parametro (name) rappresenta il nome da assegnare al cookie in cui salvare il sessionID
// The default value is **`'connect.sid'`**.

Una volta creato ed inizializzato l'oggetto `session`, le variabili session vengono memorizzate nella RAM del server di back end in modo trasparente. E' possibile rendere persistenti le variabili SESSION salvandole su un file oppure all'interno di un DB. In questo modo non vanno perse in caso di riavvio del server e soprattutto sono più facilmente gestibili nel caso di server distribuiti.

Sintassi per l'utilizzo delle variabili session

In ogni caso la variabili SESSION risultano accessibili al programmatore sempre allo stesso modo, cioè tramite l'oggetto **`session`** dell'oggetto **`request`** :

`request.session`

Creazione di una variabie / Assegnazione di un valore

```
req.session.name = 'Napoleon';
```

Lettura di un valore:

```
let name = req.session.name;
```

Cancellazione di una variabile

```
delete req.session.name
```

Rimozione dell'oggetto session

```
req.session.destroy();
```

Questo metodo accetta come parametro una eventuale funzione di callback

```
req.session.destroy(function() {
    res.send('Session deleted') });
```

Tempo di durata della sessione

```
// This user won't have to log in for a year
req.session.cookie.maxAge = 365 * 24 * 60 * 60 * 1000;
// This user should log in again after restarting the browser
req.session.cookie.expires = false;
```

I cookies con durata 0 (oppure `expires:false`) non vengono memorizzati dal browser e dunque vengono rimossi ma SOLO quando il browser viene chiuso.
Per far decadere subito il cookie occorre impostare un tempo piccolissimo (min 1 msec).

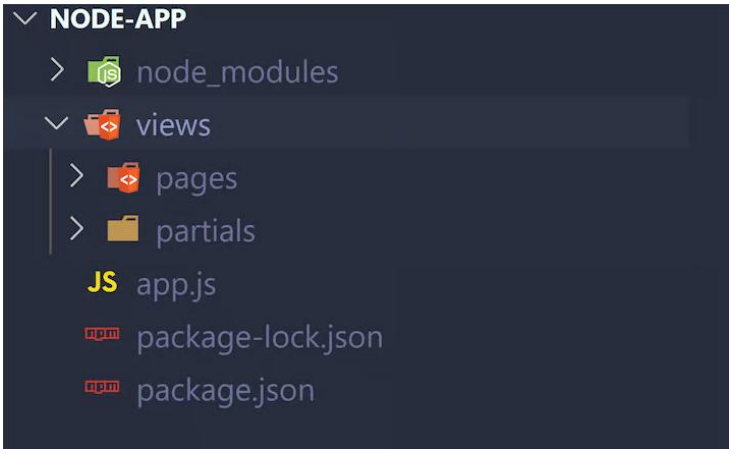
Approccio SSR ed utilizzo del template EJS

Invece di operare totalmente per servizi nella cosiddetta modalità **CLIENT SIDE RENDERING** in cui Il server invia inizialmente una pagina html statica priva di dati e poi tutta la costruzione della pagina è demandata al client, anche in Express, così come in PHP, è possibile utilizzare un approccio di tipo SERVER SIDE RENDERING in cui il server elabora i dati prima di restituire la pagina al client e poi restituisce la pagina completa con i suoi dati.

EJS (`Embedded JavaScript Templating`) è un **templating engine** (motore di rendering)**,** simile a jade, entrambi molto popolari in JavaScript. Lo scopo è quello di incorporare codice JavaScript all'interno di un modello preconfezionato per poter generare codice finale HTML lato server, esattamente come in PHP.

```
npm install ejs
```

Nella cartella di lavoro creare manualmente le seguente alberatura:



L'applicazione non sarà una applicazione TypeScrypt ma una applicazione JavaScript nativo.
Occore quindi NON importa il file tsconfig.json e la sottocartella .vscode

All'interno della sottocartella **pages** creare un **`template`** (modello) **index.ejs**

All'interno del file **app.js** (alternativa a **server.js**) inserire il seguente codice:

```
const express = require('express')
const app = express()
const port = 1337

app.set('view engine', 'ejs')

app.listen(port, () => {
  console.log(`App listening at port ${port}`)
})

app.get('/', (req, res) => {
    res.render('pages/index')
})
```

cioè:

- si imposta **EJS** come motore di rendering per la applicazione corrente,
  EJS utilizzerà come cartella di default la sottocartella '**views**'
- si avvia la app express, ma si potrebbe utilizzare ed avviare un HTTP Sever come si fa abitualmente.
- si redirige la richiesta **/** verso il file `/pages/index.ejs` (con estensione aggiunta automaticamente) in cui si può inizialmente inseire un contenuto html di prova: `<h1>Hi, there!</h1>`

- avviare l'applicazione tramite **`node app`**
- richiedendo sul brower `localhost:1337` arriverà in risposta il contenuto html di `index.ejs`

Il metodo res.render()

EJS rende disponibile all'interno dell'oggetto **`response`** un metodo res.render() che consente di richiamare un file .ejs il quale costuirà dinamicamente la pagina html da ritornare al client. Accetta due parametri:

1. Il file.ejs da caricare
2. Un json di parametri aggiunvi

```
const user = {
    firstName: 'Tim',
    lastName: 'Cook',
}

app.use('/', (req, res) => {
    res.render('pages/index', {user} )
})

<h1> Hi, I am <%= user.firstName  %> </h1>
```

`<%=` esattamente come **`<?php`** , apre un ***script*** in cui EJS andrà a valorizzare la variabili passate come paramero a res.render()

`<%=` produce un **`escaped-output`** nel senso che, se la stringa contiene dei caratteri html speciali come < > & qusti vengono automaticamente sostituiti dalla relativa codifica html, come la funzione escapeHtml(). Assolutamente raccomandato ad esempio per filtrare gli input dell'utente e proteggere l'applicazione da possibili attacchi.

Gli script EJE consentono di inserire delle sezioni di codice intercalate in qualsiasi punto della pagina html esattamente come gli script php:

```
<h1>Hi, I am <%= user.firstName  %></h1>
<% if (user.admin) { %>
    <p>Let me tell you a secret: <b>I am an admin</b></p>
<% } %>
```

Pagine Multiple

Ovviamente all'interno della app si possono scrivere più pagine EJS:

```
app.get('/', (req, res) => {
    res.render('pages/index', {user})
})
app.get('/articles', (req, res) => {
    res.render('pages/articles', {
        "articles": articolesList
    })
})

<ul>
   <% articles.forEach((article)=> { %>
      <li>
          <h2> <%= article.title %> </h2>
          <p>  <%= article.body %>  </p>
      </li>
   <% }) %>
</ul>
```

EJS Partials

Esattamente come in PHP anche in EJS è possibile includere in più pagine una medesima sezione di codice comune. Le sezioni comuni vengono di solito inserite all'interno di apposita sottocartella partials già creata in fase di inizializzazione del progetto. Si tratta comunque di una scelta arbitraria

Ad esempio l'intera sezione di head contenente link e metatag necessari in tutte le pagine, può essere inserita all'interno di un modello ***head.ejs*** posizionato all'interno di partials.
Allo stesso modo un eventule footer comune a tutte le pagine potrà essere inserito all'interno di un modello ***footer.ejs*** sempre posizionato all'interno di partials

A questo punto le varie pagine potranno essere strutturate non modo seguente:

```
<%- include('../partials/head') %>
     <!-- real html content -->
<%- include('../partials/footer') %>
```

Notare che con l'include si utilizza in genere un carattere di script diverso dal precedente.
<%- è detto **unescaped output** utilizzato per sezioni che contengono puro html.
include() semplicemente sostituisce la riga corrente con l'intero contentuo del file indicato

Le EJS partials possono accedere a tutte le variabili della parent view.
Ad esempio le due precedenti routes potrebbero essere riscritte nel modo seguente:

```
app.get('/', (req, res) => {
    res.render('pages/index', {user, "title":"Home Page"})
})
app.get('/articles', (req, res) => {
    res.render('pages/articles', {
        "articles": articolesList
        "title":"Articles"
    })
})
```

A questo punto all'interno del partials ***head.ejs*** si potrebbe scrivere una cosa del tipo

```
<title>  <%= title %>   </title>
```

La variabile `title` dovrà però essere passata a tutte le pagine che utilizzano la partials ***head.ejs***.
Attenzione che in caso di variabile undefined viene generata una eccesione.
Per proteggere l'applicazione da questo tipo di eccezioni si può utilizzare il seguente codice:

```
<title>
     <%= typeof title != 'undefined' ? title : 'Default Title' %>
</title>
```

Infine è anche possibile passare la variabile **title** direttamente al momento dell'include

```
<%- include('../partials/head', {"title":'Page Title'}) %>
```

Le variabili passate all'include prevalgono su eventuali variabili omonime passate tramite la parent view.